FastAPI — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction and Installation

02 First API

03 Path and Query Parameters

04 Request Bodies with Pydantic

05 Response Models and Status Codes

06 Handling Errors

Intermediate (6 topics)

07 Validation and Field Constraints

08 Dependency Injection

09 Databases with SQLAlchemy

10 Routers and Project Structure

11 Async Endpoints

12 Background Tasks

Advanced (6 topics)

13 Authentication: OAuth2 and JWT

14 Middleware and CORS

15 File Upload and Streaming

16 WebSockets

17 Testing

18 Settings and Configuration

Master (4 topics)

19	Performance and Caching
20	Docker and Deployment
21	Serving ML Models
22	Real Project: Blog / Task API with Auth

Beginner Level

01. Introduction and Installation

FastAPI is a modern, fast Python web framework for APIs built on type hints, Pydantic and Starlette, with automatic docs.

```
pip install "fastapi[standard]"
fastapi dev main.py
```

02. First API

Create an app and path operations with decorators; docs at `/docs` (Swagger) and `/redoc`.

```
from fastapi import FastAPI
app = FastAPI()

@app.get('/')
def root():
    return {'message': 'Hello FastAPI'}
```

03. Path and Query Parameters

Typed parameters are validated automatically.

```
@app.get('/items/{item_id}')
def item(item_id: int, q: str | None = None):
    return {'id': item_id, 'q': q}
```

04. Request Bodies with Pydantic

Pydantic models validate JSON bodies and generate schemas.

```
from pydantic import BaseModel
class Item(BaseModel):
    name: str
    price: float

@app.post('/items')
def create(item: Item):
    return item
```

05. Response Models and Status Codes

Control output shape with `response_model` and set codes with `status_code`.

```
@app.post('/users', response_model=UserOut, status_code=201)
```

06. Handling Errors

Raise `HTTPException` with status and detail; add custom exception handlers.

```
from fastapi import HTTPException
raise HTTPException(status_code=404, detail='Item not found')
```

Intermediate Level

07. Validation and Field Constraints

`Field`, `Query`, `Path` add rules like `min_length`, `gt`, regex patterns.

```
name: str = Field(min_length=3, max_length=50)
```

08. Dependency Injection

`Depends` shares logic (DB sessions, auth, pagination) across routes.

```
def get_db():
    db = SessionLocal()
    try: yield db
    finally: db.close()

@app.get('/users')
def users(db=Depends(get_db)): ...
```

09. Databases with SQLAlchemy

Define models, sessions and CRUD functions; use Alembic for migrations (or SQLModel).

```
class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True)
    name = Column(String)
```

10. Routers and Project Structure

Split code with `APIRouter`: routers, schemas, models, services.

```
router = APIRouter(prefix='/users', tags=['users'])
app.include_router(router)
```

11. Async Endpoints

Use `async def` with async libraries (httpx, asyncpg) for high concurrency; plain `def` runs in a thread pool.

```
@app.get('/data')
async def data():
    async with httpx.AsyncClient() as c:
        return (await c.get('https://api.example.com')).json()
```

12. Background Tasks

Run work after returning the response (emails, logs).

```python
@app.post('/signup')
def signup(bg: BackgroundTasks):
    bg.add_task(send_email, 'a@b.com')
```

Advanced Level

13. Authentication: OAuth2 and JWT

Password hashing (bcrypt), OAuth2PasswordBearer and JWT tokens; protect routes with dependencies.

```
oauth2 = OAuth2PasswordBearer(tokenUrl='token')
@app.get('/me')
def me(token: str = Depends(oauth2)): ...
```

14. Middleware and CORS

Add cross-cutting behaviour and allow browser clients.

```
app.add_middleware(CORSMiddleware, allow_origins=['https://myapp.com'], allow_methods=['*'])
```

15. File Upload and Streaming

Use `UploadFile` for uploads and `StreamingResponse` for large downloads.

```
@app.post('/upload')
async def up(file: UploadFile): return {'name': file.filename}
```

16. WebSockets

Real-time communication with `@app.websocket`.

```
@app.websocket('/ws')
async def ws(sock: WebSocket):
    await sock.accept()
    await sock.send_text('hi')
```

17. Testing

Use `TestClient` with pytest; override dependencies for tests.

```
client = TestClient(app)
assert client.get('/').status_code == 200
```

18. Settings and Configuration

Use `pydantic-settings` to read environment variables safely.

```
class Settings(BaseSettings):
    database_url: str
```

Master Level

19. Performance and Caching

Async DB drivers, Redis caching, pagination, connection pooling, running with multiple Uvicorn workers.

```
fastapi run --workers 4
```

20. Docker and Deployment

Containerise with a slim Python image, deploy behind Nginx/ cloud services, configure health checks.

```
FROM python:3.12-slim
COPY . /app
RUN pip install -r /app/requirements.txt
CMD ["fastapi", "run", "/app/main.py", "--port", "80"]
```

21. Serving ML Models

Load a model once at startup (lifespan) and expose prediction endpoints with validated inputs.

```
@app.post('/predict')
def predict(x: Features):
    return {'result': model.predict([x.values])[0]}
```

22. Real Project: Blog / Task API with Auth

Full CRUD API with users, JWT, PostgreSQL, Alembic migrations, tests and Docker.

Interview Question Bank

Q1. Why is FastAPI fast?
It uses Starlette and async I/O, and Pydantic's compiled validation.

Q2. What is Pydantic?
A data validation library that parses and validates data using Python type hints.

Q3. `def` vs `async def` endpoints?
`async def` runs on the event loop (use awaitable libraries); `def` runs in a thread pool.

Q4. What is `Depends`?
FastAPI's dependency injection for reusable logic like DB sessions and auth.

Q5. How are docs generated?
From type hints and Pydantic models into an OpenAPI schema, shown by Swagger UI and ReDoc.

Q6. How do you handle auth?
OAuth2 with password flow or JWT bearer tokens implemented as dependencies.

Q7. Flask vs FastAPI?
Flask is synchronous and minimal; FastAPI is async-first with built-in validation and docs.

Q8. What is `response_model`?
Filters and validates the response data shape.

Q9. How do you run background jobs?
`BackgroundTasks` for light work; Celery/RQ/ARQ for heavy jobs.

Q10. How do you test FastAPI?
`TestClient` plus pytest, overriding dependencies.

Capstone Project Ideas

1. Todo API with PostgreSQL
2. Blog API with JWT
3. URL shortener
4. ML prediction service
5. Chat app with WebSockets